

Coding & Solution

Date	03 July 2026
Team ID	N/A
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Coding & Solution

This section documents the quality and completeness of OptiCrop's implemented code and overall solution, demonstrating working functionality, clean code practices, and alignment with the defined requirements.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/v-arun-kumar-03/OptiCrop
Programming Language(s)	Python (backend & ML), HTML/CSS (frontend)
Framework(s) Used	Flask
Key Features Implemented	Soil/climate input form; multi-algorithm crop prediction (KNN, Logistic Regression, Decision Tree, Random Forest); K-Means exploratory clustering; model persistence via Pickle; Home/About/Find Your Crop pages
Pending / Incomplete Features	User authentication/admin dashboard, live weather API integration, downloadable PDF prediction reports — all planned as future scope, not required for the core solution
Setup / Run Instructions	1) python -m venv venv & venv\Scripts\activate (Windows) 2) pip install -r requirements.txt 3) python train_model.py (trains and saves model.pkl/scaler.pkl) 4) python app.py 5) Open http://127.0.0.1:5000 in a browser

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The project is organized into clear, separate concerns: dataset generation (data/), model training (train_model.py, notebooks/), the Flask application (app.py), and the frontend (templates/, static/). This separation makes it straightforward to retrain the model on new data or update the UI independently of the ML logic.